

452 MININET PROJECT
EREN BUDAK
231401007

Q1.

Görüldüğü üzere h3 ile h5 arasında ortalama yaklaşık 9.2 ms latency bulunmaktadır.

```
mininet> h3 ping -c 10 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=20.6 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=8.40 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=8.28 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=8.13 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=8.15 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=8.79 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=8.25 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=8.36 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=8.23 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=8.44 ms
```

Q2. h1 den h6 ya giderken de h2 den h5 e giderken de s1-s2 ve s2-s3 linkleri kullanılıyor.bundan dolayı bottleneck oluyor.

```
Client connecting to 10.0.0.5, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 35896 connected with 10.0.0.5 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.0443 sec  119 MBytes 32.1 Mbits/sec
```

```
Client connecting to 10.0.0.6, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 52128 connected with 10.0.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.6117 sec  257 MBytes 68.2 Mbits/sec
```

Q3. switch devre dışı bırakılmadan önce her client birbirine erişebiliyordu. Switch devre dışı bırakılınca h1 h2 ve h5 h6 bağlantısı hariç tüm linkler çöktü.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet> py net.get('s2').stop()
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X X X X
h2 -> h1 X X X X
h3 -> X X X X X
h4 -> X X X X X
h5 -> X X X X h6
h6 -> X X X X h5
*** Results: 86% dropped (4/30 received)
```

Q4.Linear Topology,Latency değeri 9.2 gibi makul bir değer olsa da Bottleneckten muzdarip.Q2 de gördüğümüz gibi herhangi bir hostlar arası gidiş geliş Bottleneck e uğrayabiliyor.Ayrıca tek bir switch in kopması neredeyse bütün bağlantıları kaldırdı,bu da büyük bir dezavantaj.

Q5.Yaklaşık 6.375 ms ortalama gecikme var,linear topology ye göre daha iyi bir sonuç.

```
mininet> h3 ping -c 10 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=17.8 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=5.09 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=4.48 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=8.11 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=4.09 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=4.15 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=4.70 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=4.20 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=4.14 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=6.99 ms

--- 10.0.0.5 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9011ms
rtt min/avg/max/mdev = 4.092/6.375/17.797/4.022 ms
```

Q6.Gözlemlenen total throughput yaklaşık 281 Mb/s.İlk fotoğrafta sonuçlar,ikincide ise uygulanışı var.

```
-----
Client connecting to 10.0.0.4, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 49594 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.7909 sec  267 MBytes  70.3 Mbits/sec
mininet> h3 iperf -c h5 -t 30
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 43374
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.8119 sec  264 MBytes  69.6 Mbits/sec
-----
Client connecting to 10.0.0.5, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 59558 connected with 10.0.0.5 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.5458 sec  265 MBytes  70.4 Mbits/sec
mininet> h4 iperf -c h6 -t 30
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.4 port 5001 connected with 10.0.0.2 port 49594
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.7753 sec  267 MBytes  70.4 Mbits/sec
-----
Client connecting to 10.0.0.6, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.4 port 56960 connected with 10.0.0.6 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.4862 sec  264 MBytes  70.3 Mbits/sec
```

```
mininet> h3 iperf -s &
[1] 8401
mininet> h4 iperf -s &
[1] 8409
mininet> h5 iperf -s &
mininet> h6 iperf -s &
mininet> h1 iperf -c h3 -t 30
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 43374 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.8282 sec  264 MBytes  69.6 Mbits/sec
mininet> h2 iperf -c h4 -t 30
-----
Client connecting to 10.0.0.4, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 49594 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-31.7909 sec  267 MBytes  70.3 Mbits/sec
```

Q7. Total throughput 74.7 Mbits/sec oldu.Çünkü aynı bottleneckden geçiliyor. 75 ms sınırını geçemez,60 a 14 şeklinde olmuş.

```
1 -----
2 Client connecting to 10.0.0.5, TCP port 5001
3 TCP window size: 85.3 KByte (default)
4 -----
5 [ 1] local 10.0.0.3 port 39644 connected with 10.0.0.5 port 5001
6 [ ID] Interval      Transfer    Bandwidth
7 [ 1] 0.0000-31.7368 sec  228 MBytes  60.3 Mbits/sec
```

```
1 -----
2 Client connecting to 10.0.0.5, TCP port 5001
3 TCP window size: 85.3 KByte (default)
4 -----
5 [ 1] local 10.0.0.4 port 33072 connected with 10.0.0.5 port 5001
6 [ ID] Interval      Transfer    Bandwidth
7 [ 1] 0.0000-30.4442 sec  52.4 MBytes  14.4 Mbits/sec
```

Q8. Bütün bağlantılar koptu,görselde de görüldüğü gibi. Çünkü hepsi s1 üzerinden bağlanıyordu birbirlerine.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet> py net.get('s1').stop()
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X X X
h2 -> X X X X X
h3 -> X X X X X
h4 -> X X X X X
h5 -> X X X X X
h6 -> X X X X X
*** Results: 100% dropped (0/30 received)
```

Q9. Değişiklik olarak 2 switch kullandık,2. Switch ilk switch koptuğunda devreye giriyor,bond arayüzü yardımıyla bir link koparsa ikincil link devreye giiriyor ve kopma sorun olmuyor.Aşağıda da sonucumuz var.Görüldüğü üzere bağlantılar hala sağlanıyor.Diğer yol olarak hostları birbirine bağlamak düşünülebilirdi.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
mininet> py net.get('s1').stop()
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6
h2 -> h1 h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
*** Results: 0% dropped (30/30 received)
```

Q10 ve Q11. Aşağıda da görüldüğü üzere switch hop sayısı 1,çünkü traceroute sadece L3 katmanını yani routerları göz önüne alıyor. Ama RTT sürelerinin azalışından pathlerdeki değişimi görebiliriz.

H1 → s1 → s2 → s3 → s4 → h4 ve h1 → s1 → s4 → h4 pathler var gibi gözüküyor.

```
mininet> link s2 s3 down
mininet> h1 traceroute h4
traceroute to 10.0.0.4 (10.0.0.4), 30 hops max, 60 byte packets
 1  10.0.0.4 (10.0.0.4) 15.173 ms 15.141 ms 15.128 ms
mininet> link s2 s3 up
mininet> h1 traceroute h4
traceroute to 10.0.0.4 (10.0.0.4), 30 hops max, 60 byte packets
 1  10.0.0.4 (10.0.0.4) 9.786 ms 9.760 ms 9.749 ms
```

Q12. S1 koptuğunda h1 in h2,h5 ve h6 ile bağlantısının koptuğunu,h2 nin de h1 ile bağlantı kuramadığını gördük.Fully connected değil artık.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 h4 X X
h2 -> X h3 h4 h5 h6
h3 -> h1 h2 h4 h5 h6
h4 -> h1 h2 h3 h5 h6
h5 -> h1 h2 h3 h4 h6
h6 -> h1 h2 h3 h4 h5
```

Q13. Normal yolda h1 den sonra sırasıyla s1 s2 s3 ve s4 den geçip h4 e varıyordu.s2 ve s3 kalkınca s1 den s4 e direk gitti,bu da latency yi düşürdü. İlki bağlantı kopmuş latency,ikincisi normal latency. Failed da ortalama 9 ms,normalde ise 17.5 ms gecikme var.

```
mininet> h1 ping -c 10 h6
PING 10.0.0.6 (10.0.0.6) 56(84) bytes of data.
64 bytes from 10.0.0.6: icmp_seq=1 ttl=64 time=11.6 ms
64 bytes from 10.0.0.6: icmp_seq=2 ttl=64 time=8.46 ms
64 bytes from 10.0.0.6: icmp_seq=3 ttl=64 time=8.26 ms
64 bytes from 10.0.0.6: icmp_seq=4 ttl=64 time=8.27 ms
64 bytes from 10.0.0.6: icmp_seq=5 ttl=64 time=8.17 ms
64 bytes from 10.0.0.6: icmp_seq=6 ttl=64 time=8.22 ms
64 bytes from 10.0.0.6: icmp_seq=7 ttl=64 time=8.18 ms
64 bytes from 10.0.0.6: icmp_seq=8 ttl=64 time=8.60 ms
64 bytes from 10.0.0.6: icmp_seq=9 ttl=64 time=8.22 ms
64 bytes from 10.0.0.6: icmp_seq=10 ttl=64 time=8.81 ms

--- 10.0.0.6 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9011ms
```

```
mininet> h1 ping -c 10 h6
PING 10.0.0.6 (10.0.0.6) 56(84) bytes of data.
64 bytes from 10.0.0.6: icmp_seq=1 ttl=64 time=24.2 ms
64 bytes from 10.0.0.6: icmp_seq=2 ttl=64 time=17.8 ms
64 bytes from 10.0.0.6: icmp_seq=3 ttl=64 time=17.2 ms
64 bytes from 10.0.0.6: icmp_seq=4 ttl=64 time=16.2 ms
64 bytes from 10.0.0.6: icmp_seq=5 ttl=64 time=16.8 ms
64 bytes from 10.0.0.6: icmp_seq=6 ttl=64 time=16.9 ms
64 bytes from 10.0.0.6: icmp_seq=7 ttl=64 time=17.3 ms
64 bytes from 10.0.0.6: icmp_seq=8 ttl=64 time=16.3 ms
64 bytes from 10.0.0.6: icmp_seq=9 ttl=64 time=17.0 ms
64 bytes from 10.0.0.6: icmp_seq=10 ttl=64 time=16.4 ms

--- 10.0.0.6 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 16.191/17.595/24.181/2.243 ms
```

Q14.

Traceroute switchleri dikkate almaz,hoplar 1.Tutarlılığın sebebi ise genel olarak spine leaf in yapısıyla alakalı.Sadece 1 leafe uğrayanlar daha kısa sürmüş(h3-h4,h1-h2 gibi),leaf-spine-leaf yapanlar ise daha uzun sürmüş,yollar arasında tutarlılık var(h1-h3 ve h2-h3 örnektir uzun sürenlere).

```
mininet> h1 traceroute -n h6
traceroute to 10.0.0.6 (10.0.0.6), 30 hops max, 60 byte packets
 1  10.0.0.6  16.336 ms  16.315 ms  16.309 ms
mininet> h2 traceroute -n h3
traceroute to 10.0.0.3 (10.0.0.3), 30 hops max, 60 byte packets
 1  10.0.0.3  13.443 ms  13.420 ms  13.412 ms
mininet> h1 traceroute -n h2
traceroute to 10.0.0.2 (10.0.0.2), 30 hops max, 60 byte packets
 1  10.0.0.2  4.578 ms  4.553 ms  4.545 ms
mininet> h3 traceroute -n h4
traceroute to 10.0.0.4 (10.0.0.4), 30 hops max, 60 byte packets
 1  10.0.0.4  4.269 ms  4.249 ms  4.241 ms
mininet> h5 traceroute -n h6
traceroute to 10.0.0.6 (10.0.0.6), 30 hops max, 60 byte packets
 1  10.0.0.6  4.224 ms  4.339 ms  4.331 ms
mininet> h1 traceroute -n h3
traceroute to 10.0.0.3 (10.0.0.3), 30 hops max, 60 byte packets
 1  10.0.0.3  14.480 ms  14.459 ms  14.452 ms
```

Q15. Spine Leaf Lineer ve Star a göre daha kötü bir ortalamada kaldı.Consistent ama leaf spine leaf gibi uzun yollar çok fazla.Star ise en hızlı olan 6.4 ms ile.Lineer ise 9.2 ms latency ile 2. sırada.

```
mininet> h3 ping -c 10 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data:
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=13.0 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=13.1 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=12.1 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=12.2 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=12.1 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=12.1 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=13.8 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=12.5 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=12.2 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=12.3 ms
--- 10.0.0.5 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9011ms
rtt min/avg/max/mdev = 12.102/12.543/13.836/0.567 ms
```

Q16. Aynı seviyelerde bandwidth, sebebi ise kullanılan spine in spine 2 olması. Spine 1 in durdurulması hiç bir kayda değer değişiklik görülüyor.

```
mininet> iperf h1 h4
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['69.3 Mbits/sec', '69.1 Mbits/sec']
mininet> py net.get('spine1').stop()
mininet> iperf h1 h4
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['71.0 Mbits/sec', '70.8 Mbits/sec']
```

Q17.

Type	Switches	Links
Linear	3	8
Star	1	6
Ring	6	9
Spine Leaf	5	13

En düşük maliyet star, en yüksek ise Spine leaf de.

Q18.

Type	Cost	Latency(ms)	Connectivity after failure
Linear	$3+8=11$	~9.2 ms	Nerdeyse tüm bağlantılar s1 kopunca gitti.
Star	$1+6=7$	6.375 ms	S1 koparsa tüm bağlantı gider.
Ring	$6+9=15$	13.008 ms	S3 failure da h3 kopar.
Spine Leaf	$5+13=18$	12.543 ms	Fully Connected
Star Version 2	$2+13=15$	4.333 ms	Fully Connected

Q19.

Test A da bağımsız flowlar var. Bu yüzden teorik olarak $3 \times 75 = 225$ e kadar total aggregate throughput olabilirdi. Nitekim 211.10 çıktı Throughput.

Test B de aynı spine üzerinden geçen 2 flow var, bottleneck kaynaklı total 100 Mbps yi geçemezler. Nitekim total 93.9 throughput çıktı.

```
===== Test A (balanced across leaves) =====  
h1 -> h4: 70.30 Mbps  
h3 -> h6: 70.40 Mbps  
h5 -> h2: 70.40 Mbps  
Total aggregate throughput for Test A (balanced across leaves): 211.10 Mbps  
  
===== Test B (bottleneck on leaf1) =====  
h1 -> h5: 46.90 Mbps  
h2 -> h6: 47.00 Mbps  
Total aggregate throughput for Test B (bottleneck on leaf1): 93.90 Mbps
```

Q20.

a-)Spine Leaf kullanılabilir,latency değeri ortalama olsa da pathler açısından zengin ve kopmalar iletişimi pek etkilemiyor.Maliyeti çok ama şirket bunu göz ardı edebilir.

b-)Star Topology için yaptığımız 2 switchli versiyon maliyet olarak biraz daha düşük ve özellikle Latency de çok iyi,özellikle bir switch kaybına sıkıntı çıkarmaması önemli.